

TDD · BDD · Spec-Driven

Development in the Agentic Era — one MCP-powered workflow, hands on

60-minute workshop · bring your laptop · you drive an agent from exercise one

RED

GREEN

REFACTOR

— now with agents

What you will leave with

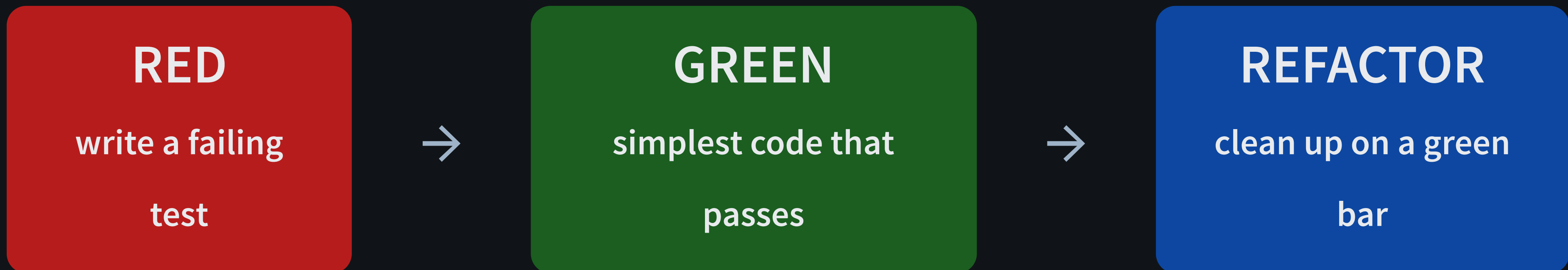
- A **spec-authoring loop**: you and an AI agent draft a requirement, `validate_spec` checks its structure, `refine_requirement` critiques its wording — and the agent iterates until both are clean
- A complete **spec-to-green workflow** you drove yourself: requirement → Gherkin → RED → GREEN → refactor
- A working grasp of how MCP feeds your tools into **any agent** — Cursor, Claude, or the bundled CLI client

The next 60 minutes

agenda

- | | |
|-------|--|
| 0–5 | Welcome, setup check (branch off trunk; <code>mvn -q package && mvn -q -f kata/pom.xml test</code>) |
| 5–13 | TDD refresher, agents, and the SDD → BDD → TDD stack |
| 13–20 | The plumbing: MCP in seven minutes — the pre-built server and client, and how tools reach any agent |
| 20–32 | EX 1 Draft the spec with your agent, iterate with <code>validate_spec / refine_requirement</code> |
| 32–52 | EX 2 Spec → Gherkin → Red/Green/Refactor, agentically |
| 52–55 | Why this works reliably |
| 55–60 | Takeaways, where to go next, Q&A |

Refresher: the discipline that scales



- The test is written *first* — it is the executable specification
- Never refactor on a red bar
- Small steps, fast feedback, permanent safety net

What changes in the agentic era?

Stays the same

- Red/Green/Refactor cycle
- Tests define "done"
- Human owns every engineering decision

Gets amplified

- **Spec authoring:** you describe intent, the agent drafts the requirement, tools validate and critique it
- **Requirements → tests:** agents draft failing tests from acceptance criteria
- **Green phase:** a developer agent proposes the implementation
- **Feedback loop:** tools report bar color back to the agent automatically

The agent turns the crank. You steer.

Three altitudes, one discipline

	Pins	Canonical artifact	In this repo
SDD spec-driven	the feature	versioned spec + acceptance criteria	requirements/requirements.js
BDD behavior-driven	one behavior	Gherkin scenario (Given/When/Then)	kata/src/test/resources/feat
TDD test-driven	one unit	failing JUnit test	kata/src/test/java/.../Strin

- Same "specify first" idea at three levels — they **stack**, they don't compete
- Agentic SDD: the spec is the source of truth; agents generate scenarios, tests, and code *from* it
- Our `run_tests` tool runs Cucumber + JUnit together: one bar, one color

TDD pins the unit. BDD pins the behavior. SDD pins the feature.

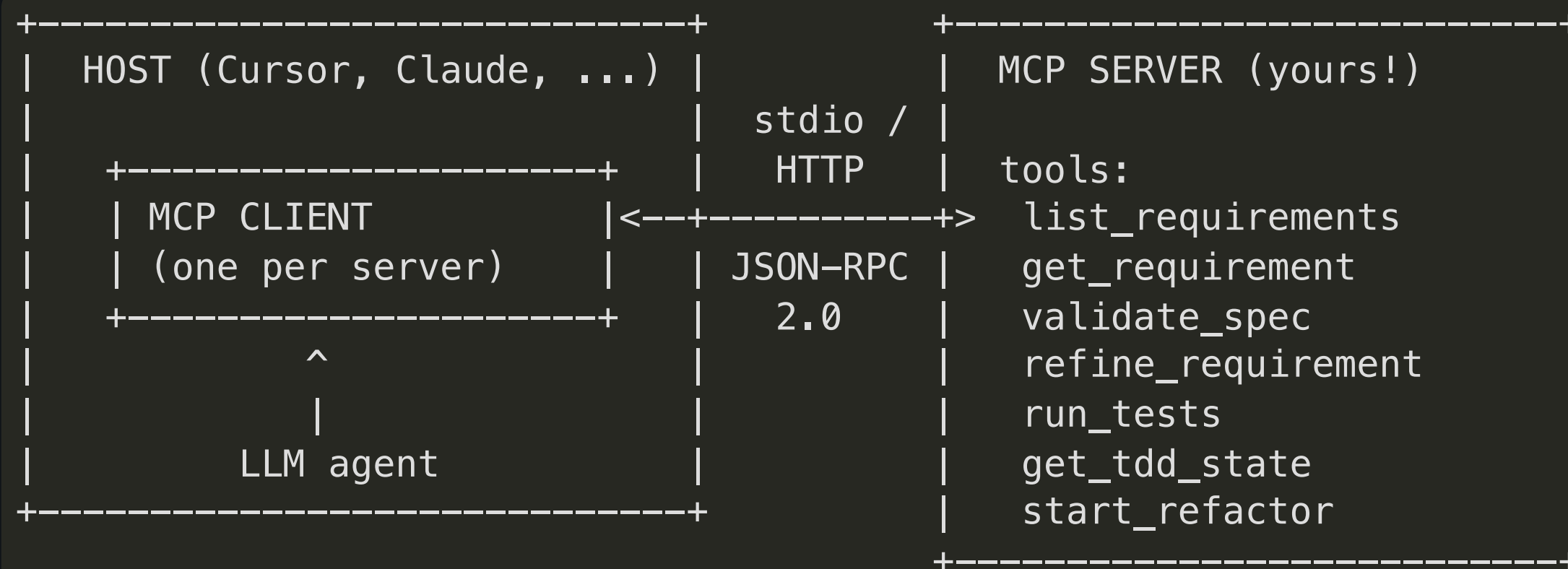
The missing piece: a protocol

For an agent to run *your* tests, read *your* requirements, and respect *your* workflow, it needs a standard way to discover and call your tools.

That standard is the **Model Context Protocol**.

Think "USB-C for AI tooling": one connector, any host (Cursor, Claude, your own client), any tool.

MCP architecture



- **Host** embeds an LLM and one MCP **client** per connection
- **Server** exposes *tools*, *resources*, and *prompts*
- Transports: **stdio** (today) or Streamable HTTP

The wire: JSON-RPC 2.0

Three message types: requests (have an `id`), responses, notifications (no `id`).

```
{ "jsonrpc": "2.0", "id": 1, "method": "initialize",  
  "params": { "protocolVersion": "2025-11-25",  
    "capabilities": { },  
    "clientInfo": { "name": "tdd-workshop-agent", "version": "1.0.0" } } }
```

```
{ "jsonrpc": "2.0", "id": 1,  
  "result": { "protocolVersion": "2025-11-25",  
    "capabilities": { "tools": { "listChanged": true }, "logging": { } },  
    "serverInfo": { "name": "tdd-workflow-server", "version": "1.0.0" },  
    "instructions": "Drives a spec-driven TDD/BDD workflow..." } }
```

Lifecycle of every MCP session

1. `initialize` — version + capability negotiation
2. `notifications/initialized` — client says "ready"
3. `tools/list` — **discovery**: what am I allowed to do?
4. `tools/call` — **invocation**: do it
5. Repeat 3–4; notifications flow both ways (logging, list changes)

Discovery is the superpower: the agent adapts to whatever tools the server offers — no hardcoding.

PLUMBING

The server — already built

13–20 min

mcp-server/ is complete, spec-driven, and tested to 100% coverage. Two minutes on its shape:

```
// TddMcpServer.main — the whole composition root
WorkflowToolHandlers handlers = new WorkflowToolHandlers(
    () -> RequirementsRepository.load(requirementsFile), // re-read fresh per call
    new SpecValidator(requirementsFile, root),           // structure: is it usable?
    new RequirementRefiner(),                             // wording: is it any good?
    new MavenTestRunner(root),
    new TddStateMachine());

McpSyncServer server = McpServerFactory.create(
    new StdioServerTransportProvider(McpJsonDefaults.getMapper()), handlers);
```

One transport, seven tools. A stdio server must **never print to stdout** — that corrupts the JSON-RPC stream. Diagnostics go to stderr.

Anatomy of a tool — what the agent sees

```
new SyncToolSpecification(  
  Tool.builder("get_requirement", Map.of(  
    "type", "object",  
    "properties", Map.of("id", Map.of(  
      "type", "string",  
      "description", "The requirement id, e.g. REQ-003")),  
    "required", List.of("id")))  
  .description("Get the user story and acceptance criteria for one requirement. "  
    + "Turn each acceptance criterion into a failing JUnit test in "  
    + "kata/src/test/java before writing production code.")  
  .build(),  
  (exchange, request) -> handlers.getRequirement(request.arguments()))
```

- **Name + JSON Schema + description** → the API docs the LLM reads at discovery
- **Handler** → plain Java; return content + `isError`
- This is how MCP feeds *your* tools into *whatever* agent your team uses

The tool that closes the loop: `run_tests`

```
// MavenTestRunner: launch `mvn -q -B -f kata/pom.xml test`, then parse Surefire XML
TestRunSummary summary = testRunner.runKataTests();
var phase = tdd.recordTestRun(summary); // failures -> RED, all pass -> GREEN

body.put("phase", phase.name()); // the agent literally sees the bar color
body.put("failureDetails", summary.failureDetails());
body.put("nextStep", tdd.suggestion()); // RED: "Write the simplest production code..."
```

The TddStateMachine enforces the discipline in code: `start_refactor` returns `isError: true` unless the bar is GREEN ("Never refactor on a red bar"). The server is itself spec-driven — its own requirements file, Cucumber scenarios, and 100% coverage: `mvn verify -pl mcp-server`.

Proof it feeds any agent: the bundled client

2 min demo

```
ServerParameters params = ServerParameters.builder("java")
    .args("-Dworkshop.root=" + root, "-jar", serverJar.toString())
    .build();

McpSyncClient client = McpClient.sync(
    new StdioClientTransport(params, McpJsonDefaults.getMapper()))
    .requestTimeout(Duration.ofMinutes(6)) // run_tests invokes Maven
    .clientInfo(new McpSchema.Implementation("tdd-workshop-agent", "1.0.0"))
    .build();

client.initialize(); // the handshake
var tools = client.listTools(); // discovery
var result = client.callTool( // invocation
    CallToolRequest.builder("run_tests").arguments(Map.of()).build());
```

Run it: `java -jar mcp-client/target/tdd-agent.jar` — expect
"phase": "GREEN", "tests": 5

EXERCISE 1

Draft the spec with your agent

20–32 min

The spec comes first — and the agent helps write it, then *refine* it. Prompt your IDE agent:

"Add a new requirement to `requirements/requirements.json`: newlines may separate numbers in addition to commas. Follow the existing format — unique id, title, user story, acceptance criteria phrased Given/When/Then, status pending. Then call `validate_spec` and fix every issue until the spec is valid. Then call `refine_requirement` on the new requirement and reword it from the findings until there are none. Do not write scenarios or code yet — we are only agreeing on the spec."

- Two feedback loops: `validate_spec` owns **structure**, `refine_requirement` owns **wording quality**
- The agent drafts, the server critiques, the agent rewords — **you approve the final wording**

The spec iteration loop — two stages

you describe the feature in a sentence of intent

agent drafts `REQ-007` into `requirements.json` (past `REQ-006`; status stays pending)

structure `validate_spec` → "criterion must be phrased Given/When/Then" → agent fixes → `"valid": true` (a well-formed draft is often valid on the first call — force an issue to demo the loop)

wording `refine_requirement` → "'quickly' is ambiguous", "only happy paths — add an edge case" → agent rewords → validates → refines → `"clean": true`

you read the story and criteria aloud — *is this what we meant?* Approve.

The server re-reads the file on every call, so the spec the agent just wrote is the spec that gets critiqued. A valid, clean, approved spec is the **entry ticket** — no scenario or code before it.

EXERCISE 2 Spec to green, agentically

32–52 min

Wire the server into your IDE agent (Cursor shown; copy `config/mcp.json` — `.cursor/` is local-only):

```
// config/mcp.json (the committed entry; Cursor reads .cursor/mcp.json)
{
  "mcpServers": {
    "tdd-workflow": {
      "command": "java",
      "args": ["-Dworkshop.root=${workspaceFolder}",
               "-jar", "${workspaceFolder}/mcp-server/target/tdd-mcp-server.jar"]
    }
  }
}
```

Then prompt the agent:

"Using the tdd-workflow tools: validate the spec first, then find the next pending requirement, add a Gherkin scenario for its acceptance criteria to the feature file (tag it with the requirement id), reuse or add step definitions, add a matching JUnit unit test, run the tests to show RED, then implement the simplest code to reach GREEN, then refactor, then mark the requirement implemented in `requirements/requirements.json`. Ask me before each phase change."

The loop you will drive

agent

`validate_spec` → the spec (including your new requirement) is valid

agent

`get_requirement("REQ-003")` → criteria +
`featureLocation, workflowHint`

agent

writes the @REQ-003 Gherkin scenarios (BDD) and a matching @Test (TDD)

you

review the scenario — *is this the behavior I want?* (the spec review)

RED

`run_tests` → Cucumber + JUnit failures returned to the agent

agent

implements the simplest passing code in `StringCalculator`

GREEN

`run_tests` → all pass

REFACTOR

`start_refactor` → clean up → `run_tests` stays green

you

approve; agent marks REQ-003 implemented in the spec; repeat with REQ-004..006

Why this works reliably

- **Valid spec first:** `validate_spec` gates the pipeline — no scenario or code from a broken spec
- **Ground truth:** the agent doesn't guess whether code works — `run_tests` tells it
- **Guardrails in tools:** refactor-on-red is impossible; the server refuses
- **Small steps:** one requirement, one cycle — failures stay cheap
- **Portable:** same server works in Cursor, Claude, our CLI agent MCP Inspector

What you accomplished in 60 minutes

- Drafted a requirement *with* an agent and iterated it through two server feedback loops — `validate_spec` for structure, `refine_requirement` for wording — until valid and clean
- Drove a full spec-to-green cycle: requirement → Gherkin (Cucumber) → RED → GREEN → refactor
- Saw how MCP feeds the same seven tools into any agent — Cursor, Claude, or the bundled client
- Kept every engineering decision: the agent turned the crank, you steered

Keep building

- The **bdd** CLI automates this loop — same seven tools; point it at your project with `bdd mcp serve`
- Homework: REQ-004–006 still pending; take REQ-007 (the one you drafted) to green
- Grade your run: `scripts/verify-workshop-run.sh` check

Keep building

Resources

- This repo: everything you used today, AGPL-3.0 licensed
- Site, install, and CLI manual: davidparry.github.io/tdd-bdd-agentic
- MCP spec & docs: modelcontextprotocol.io
- MCP Java SDK: github.com/modelcontextprotocol/java-sdk
- MCP Inspector: `npx @modelcontextprotocol/inspector`
- Kent Beck, *Test-Driven Development: By Example* — still the book

Thank you!

Questions — and let's see your green bars.

BACKUP

Inspect the wire

if time allows

Option A — MCP Inspector (browser UI + message log):

```
npx @modelcontextprotocol/inspector \  
  java -Dworkshop.root=$PWD -jar mcp-server/target/tdd-mcp-server.jar
```

Option B — be the client yourself; paste one line at a time:

```
java -Dworkshop.root=$PWD -jar mcp-server/target/tdd-mcp-server.jar  
{  
  "jsonrpc": "2.0",  
  "id": 1,  
  "method": "initialize",  
  "params": {  
    "protocolVersion": "2025-11-25",  
    "capabilities": {},  
    "clientInfo": {}  
  }  
}  
{  
  "jsonrpc": "2.0",  
  "method": "notifications/initialized"  
}  
{  
  "jsonrpc": "2.0",  
  "id": 2,  
  "method": "tools/list"  
}  
{  
  "jsonrpc": "2.0",  
  "id": 3,  
  "method": "tools/call",  
  "params": {  
    "name": "get_tdd_state",  
    "arguments": {}  
  }  
}
```

- The protocol is newline-delimited JSON on stdin/stdout — nothing hidden
- Tool results are content blocks; `isError: true` lets agents read failures and self-correct